

Proyecto de informática I

Tema 2. Planificación y gestión de proyectos



Universidad
Europea

Tema 2. Planificación y gestión de proyectos

Índice

1. Presentación.....	3
2. Introducción a las metodologías ágiles	3
3. Objetivos de las metodologías ágiles	4
4. Los doce principios	5
5. Beneficios de las metodologías ágiles	6
6. El ciclo de vida ágil.....	7
7. Pasos del ciclo de vida ágil.....	8
8. Estimación de funcionalidades (<i>feature estimation</i>).....	10
9. Resumen	10
Referencias bibliográficas	11
Enlaces de interés	11

Tema 2. Planificación y gestión de proyectos

1. Presentación

La **gestión de proyectos *software*** es la disciplina que se encarga de la planificación y la consecución de proyectos *software*. Esta disciplina no dista, en gran medida, de la gestión de otro tipo de proyectos (arquitectura, obras públicas...). Sin embargo, se tienen en cuenta las casuísticas propias del desarrollo *software*.

En la década de los 80, la industria del *software* creció con rapidez. Para gestionar los esfuerzos de desarrollo, se empezaron a utilizar metodologías de gestión de proyectos. En poco tiempo, se dieron cuenta de que los métodos de gestión de proyectos de otras disciplinas no conseguían cumplir los requisitos del usuario (cliente) con el *software* generado. Para eliminar dichos problemas en la gestión de proyectos *software* se empezó a utilizar el **modelo en cascada**.

En principio, el modelo en cascada evita desviaciones entre los requisitos del usuario y el *software* generado. Pero, en muchas ocasiones, es difícil reunir y documentar los requisitos del cliente. En este modelo, todos los detalles del proyecto han de ser definidos desde el principio, y esto puede intimidar al cliente. Además, el cliente no siempre va a ser capaz de visualizar el resultado final del producto desde el documento de requisitos, por lo que se pueden generar conflictos en la finalización del proyecto.

La **metodología ágil** se basa en desarrollos rápidos de funcionalidades. De este modo, el cliente tiene la oportunidad de ver parte del *software* terminado y hacer cambios en base a sus impresiones, evitando el problema del modelo en cascada mencionado anteriormente.

En este tema, veremos cómo se realiza una **gestión de proyectos con metodologías ágiles**.

2. Introducción a las metodologías ágiles

Las **metodologías ágiles** son una forma de gestionar proyectos. Estas metodologías pueden servir para cualquier tipo de gestión, pero se crearon para la gestión y desarrollo de proyectos *software*.



Las metodologías ágiles dividen los proyectos en pequeñas partes más manejables, llamadas **sprints**.

Al final de cada *sprint* (el cual se desarrolla en un espacio de tiempo limitado y predecible), algo de valor es producido.

El producto generado en cada *sprint* debe de poder usarse (tener funcionalidad), de modo que pueda ser valorado por el cliente o los accionistas.

Tema 2. Planificación y gestión de proyectos

Al contrario que en la **metodología *waterfall***, que es estrictamente **secuencial**, en las metodologías ágiles, diseñadores, desarrolladores y las personas encargadas del negocio trabajan conjuntamente y al mismo tiempo.



Secuencial

No se puede empezar el diseño hasta que la totalidad de los requisitos son conocidos, y tampoco se puede empezar el desarrollo hasta que todo el diseño está terminado.

3. Objetivos de las metodologías ágiles

Como indica el **Manifiesto Ágil**, en un proyecto ágil se valora:

1. Los **individuos y sus interconexiones** son más importantes que los procesos y herramientas.
2. El **software funcional** es más importante que la documentación.
3. La **colaboración con el cliente** es más importante que las especificaciones contractuales.
4. La **capacidad de cambio** es más importante que seguir un plan estricto.

Las metodologías ágiles tienen en cuenta que los proyectos *software* -y el *marketing*- son inherentemente impredecibles.

En el transcurso de un proyecto es de esperar que haya cambios. Estos cambios pueden ser por el **cambio del mercado** (por ejemplo, la funcionalidad añadida por la competencia) o cambios producidos por **decisiones del cliente**.



Esta metodología se creó para tratar esta **impredecibilidad**: dividiendo los proyectos en funcionalidades concretas y autocontenidas, siendo así muy fácil añadir, eliminar o cambiar la prioridad de las funcionalidades.

Esto sería completamente imposible con un **modelo en cascada**, en el que hasta que no se completa una fase, es imposible pasar a la siguiente.

4. Los doce principios



Según el **Manifiesto Ágil**, todo proyecto ágil debe respetar los doce principios.

El enunciado de cada uno de los **doce principios** es el siguiente:

1. Nuestra mayor prioridad es **satisfacer al cliente** mediante la entrega temprana y continua de *software* con valor.
2. **Aceptamos que los requisitos cambien**, incluso en etapas tardías del desarrollo. Los procesos ágiles aprovechan el cambio para proporcionar ventaja competitiva al cliente.
3. Entregamos ***software* funcional** frecuentemente, entre dos semanas y dos meses, con preferencia al periodo de tiempo más corto posible.
4. Los responsables de negocio y los desarrolladores **trabajamos juntos** de forma cotidiana durante todo el proyecto.
5. Los proyectos se desarrollan en torno a **individuos motivados**. Hay que darles el entorno y el apoyo que necesitan y confiarles la ejecución del trabajo.
6. El método más eficiente y efectivo de comunicar información al equipo de desarrollo, y entre sus miembros, es la **conversación cara a cara**.
7. El ***software* funcionando** es la medida principal de progreso.
8. Los procesos ágiles promueven el **desarrollo sostenible**. Los promotores, desarrolladores y usuarios debemos ser capaces de mantener un ritmo constante de forma indefinida.
9. La **atención continua** a la excelencia técnica y al buen diseño mejora la agilidad.
10. La **simplicidad**, o el arte de maximizar la cantidad de trabajo no realizado, es esencial.
11. Las mejores arquitecturas, requisitos y diseños emergen de equipos **autoorganizados**.
12. A intervalos regulares, el equipo reflexiona sobre **cómo ser más efectivo** para, a continuación, ajustar y perfeccionar su comportamiento en consecuencia.

5. Beneficios de las metodologías ágiles

- **Inmediatez de resultados.** La metodología ágil permite presentar tu concepto del producto tan rápido como sea posible. En cada *sprint* (desarrollo de una funcionalidad), un proyecto ágil entrega algo de valor. Al terminar cualquier *sprint* puedes decidir publicar una versión del *software*, comprobar la reacción del mercado y tomar decisiones.
- **Flexibilidad.** La metodología ágil se basa en amoldarse al cambio. Conforme el producto se lanza al mercado, puede requerir cambios rápidos (por ejemplo, adaptarse a la competencia). El equipo de desarrollo debe ser capaz de reaccionar y lanzar una nueva versión del producto. Aparte, esta metodología permite que ideas que surgen a mitad del desarrollo puedan llevarse a cabo.
- **Menor riesgo.** El lanzamiento de versiones incremental significa que el producto puede ser usado por el cliente en una fase temprana (con la mínima funcionalidad). Esto permite que el cliente indique la funcionalidad que es prioritaria, adaptando el proyecto a estos nuevos requisitos. Con otras metodologías, este planteamiento sería imposible.
- **Control de costes.** La metodología ágil es flexible en cuanto a la funcionalidad a desarrollar. Es común que los clientes se den cuenta de que querían una funcionalidad que, finalmente, no era necesaria. Esto les permite tener el producto antes y pagar menos. Así, esta metodología permite pagar solo por lo que finalmente necesitas.

Y, ¿si hay que ajustarse a un presupuesto? Sin problema, se prioriza la funcionalidad más importante dejando la funcionalidad secundaria fuera del proyecto.

- **Calidad.** La metodología ágil integra las pruebas de cada funcionalidad. Esto implica que cada versión del *software* esté completamente testeada. En otras metodologías, toda la funcionalidad ha de ser testeada en el último paso, siendo esto más engorroso.
- **Producto mejor.** El desarrollo incremental de versiones permite probar el producto al final de cada iteración. El público puede dar información sobre el producto, siendo más fácil localizar los puntos flojos y los elementos que se pueden mejorar cuando tienes un producto que realmente funciona, en comparación a cuando solo tienes la documentación de diseño.



6. El ciclo de vida ágil

Hay muchas versiones de las metodologías ágiles, aunque la mayoría de los conceptos son universales. Generalmente, todos ellos siguen un corto ciclo de vida (ya que es en lo que se basan estas metodologías) que se repite en cada iteración. Aquí, utilizaremos *scrum*.



Los **proyectos scrum** se dividen en iteraciones cortas (normalmente, entre una y tres semanas), llamadas **sprints**.

Antes de pasar a describir el ciclo de vida de un **sprint**, es necesario conocer ciertos conceptos.



El ciclo de vida de cada **sprint** incluye:

- Planificación.
- Ejecución.
- Revisión.
- “Aclarar y repetir”.

- **Planificación inicial (*kickoff planning*)**

Cada proyecto *scrum* se inicia con una **planificación inicial**.

Esta primera reunión, generalmente, dura más, ya que se tiene que crear la **lista de trabajos pendientes (*project backlog*)** y el equipo presentado.

- **Planificación de sprint (*sprint planning*)**

Esta es una reunión que se realiza al principio de cada *sprint*.

El equipo se reúne para decidir el objetivo del *sprint* y qué subconjunto de **trabajos pendientes (*backlog*)** va a ser desarrollado.

Por cada *sprint* se creará un ***sprint backlog***.

- **Lista de trabajos pendientes (*project backlog*)**

Todo proyecto *sprint* consta de un ***backlog***.

El ***backlog*** es una lista de las funcionalidades del producto, generalmente, definidas como “**historias de usuario**” (*user stories*).

Tema 2. Planificación y gestión de proyectos

Comúnmente siguen la siguiente estructura: **COMO [TIPO DE USUARIO]** quiero **[HACER ALGO]** de forma que pueda **[OBJETIVO A CONSEGUIR]**.

- ***Sprint backlog***

El ***sprint backlog*** se genera al principio de cada *sprint* y se actualiza durante la ejecución del *sprint*.

Su utilidad es ayudar al equipo a planificar y gestionar el trabajo necesario para alcanzar el objetivo del *sprint*.

7. Pasos del ciclo de vida ágil

Paso 1. *Sprint planning*

El trabajo a realizar en cada *sprint* del proyecto se planifica en el ***sprint planning***. En la creación de este plan colabora todo el equipo *scrum*. En esta reunión se decide:

- Qué resultado puede ser entregado tras la finalización del *sprint*.
- Cómo se va a realizar el trabajo necesario para completar la nueva funcionalidad.

La funcionalidad a implementar se selecciona del ***project backlog***. Esta funcionalidad será la base para generar el ***sprint backlog***.



Cómo mantener un *project backlog*:

Hay muchas herramientas para mantener el ***project backlog***: existen opciones **analógicas y digitales**. Lo importante es que siempre esté accesible y sea fácil de leer. La forma más básica sería mantener *post-it* en una pared, ya que son fáciles de mover a otras posiciones.

Existen varias soluciones para tener el ***project backlog*** en línea (por ejemplo, *Trello*). Tras la **planificación inicial**, todas las **historias de usuario** han sido añadidas. Acto seguido, se ordenan según su prioridad. Dentro de esta ordenación, se pueden agrupar varias historias en una sola.

Recuerda que el ***project backlog*** es siempre fluido y no cerrado. El líder del proyecto puede cambiar las prioridades de las historias entre *sprints*. También se pueden añadir nuevas historias según avanza el proyecto.

El ***project backlog*** se mantendrá cerrado mientras se ejecuta un *sprint*. Mientras se ejecuta un *sprint*, es importante mantener el ***project backlog*** cerrado para mantener al equipo centrado en el *sprint* actual.

Tema 2. Planificación y gestión de proyectos

Paso 2. *Sprint execution*

La **ejecución** de un *sprint*, normalmente, dura entre tres o cuatro semanas, aunque no hay una norma fija.

Durante un *sprint*, diseñadores, desarrolladores y gestores trabajan conjuntamente.

Los objetivos y el trabajo a realizar están marcados en el ***sprint backlog***. Mientras dure el *sprint*, no se pueden añadir nuevas funcionalidades, y los objetivos del *sprint* no pueden cambiar.



Reunión diaria (*daily standup*)

Durante la ejecución de un *sprint*, la primera actividad de cada día es una **reunión** de no más de 15 minutos que incluye a todo el equipo.

Cada miembro del equipo debe responder a las siguientes preguntas:

- ¿Qué hiciste ayer?
- ¿Qué vas a hacer hoy?
- ¿Necesitas ayuda o hay algo que te bloquee continuar?

El resultado de esta reunión, normalmente, se comenta con el cliente.

Esta reunión permite averiguar si algo está retrasando el trabajo, para poder responder rápidamente ante tal evento.



Paso 3. *Sprint review meeting*

Al final de cada *sprint*, algo con valor (una nueva funcionalidad) es añadido al producto.

En el ***sprint review meeting*** están convocados los miembros del equipo *scrum* y el cliente. El motivo de esta reunión es mostrar el trabajo completado. Esta reunión siempre empieza con una demo funcional. Se comenta cómo se puede mejorar y se revisan las prioridades del ***project backlog***.

A continuación, se pasa a la fase de ***sprint planning***.



Tema 2. Planificación y gestión de proyectos

8. Estimación de funcionalidades (*feature estimation*)



Para estimar qué puede ser hecho en cada *sprint* y cuánto tiempo va a llevar desarrollar todo el proyecto, es necesario estimar cuánto tiempo llevará cada **historia de usuario** (*user story*).

Uno de los mayores desafíos en desarrollo es predecir con precisión el tiempo necesario para llevar la tarea a cabo, debido a la dificultad en la estimación.

Las **funcionalidades** se valoran en una escala con puntos del uno al tres.

- Si intentamos hacer una escala más precisa, será más difícil darle el valor correcto y, además, menos preciso.
- Si hay alguna funcionalidad que se sale de la escala (mayor que tres), se debería dividir en funcionalidades más pequeñas que sí quepan en la escala.



También, es importante determinar la **velocidad del equipo *scrum***, o lo que es lo mismo, cuántos “puntos” puede completar el equipo en cada *sprint*.

Esta velocidad se conoce con el tiempo: cuánta más experiencia tienes con un equipo, más precisa se hará esta medida.

9. Resumen

La **metodología ágil** está pensada para facilitar el desarrollo, no para complicarlo.

- Esto quiere decir que esta metodología te ayudará a mejorar el tiempo de publicación del *software* generado y por menos dinero.
- Está pensada para ser menos arriesgada que la metodología *waterfall*.
- También permite a los equipos trabajar juntos para aprovechar lo mejor de cada uno y evitar que el trabajo se atasque en algún punto del proceso.

Tema 2. Planificación y gestión de proyectos

Referencias bibliográficas

Rasmusson, Jonathan. (2017). *The agile samurai. How agile masters deliver great software*. The pragmatic programmers.

Pro Git. <www.git-scm.com/book/en/v2>

Enlaces de interés

Manifesto for agile software development. <<http://agilemanifesto.org/>>

Linkedin. <<https://www.linkedin.com/groups/37631/profile>>

The lean startup. <<http://theleanstartup.com/>>

Trello. <<https://trello.com/>>

Try GitHub. <www.try.github.io/levels/1/challenges/1>

Wikibooks. <https://en.wikibooks.org/wiki/Java_Swings/Java_Swing>

Oracle. <<https://docs.oracle.com/javase/tutorial/uiswing/TOC.html>>



**Universidad
Europea**

© Todos los derechos de propiedad intelectual de esta obra pertenecen en exclusiva a la Universidad Europea de Madrid, S.L.U. Queda terminantemente prohibida la reproducción, puesta a disposición del público y en general cualquier otra forma de explotación de toda o parte de la misma.

La utilización no autorizada de esta obra, así como los perjuicios ocasionados en los derechos de propiedad intelectual e industrial de la Universidad Europea de Madrid, S.L.U., darán lugar al ejercicio de las acciones que legalmente le correspondan y, en su caso, a las responsabilidades que de dicho ejercicio se deriven.